MicroROM-NPU

RISC-V ISA Selection & Datapath Width — Design Note

1. Why RISC-V was chosen

RISC-V provides a documented, open instruction-set architecture that we can implement directly in our own standalone processor-like NPU engine. This gives the project practical exposure to instruction decoding, register operations, control flow, datapath design and microsequencer control, without adopting an existing CPU core.

For this version, the plan is to use standard RV32I instruction semantics and standard RV32C compressed encodings where supported. Eligible instructions can be 16 bits; uncompressed base instructions are 32 bits. We are not inventing a custom 16-bit encoding.

Scope: This is a purpose-built implementation of a selected instruction subset, not a claim of full RV32I/RV32C compliance. Internal MicroROM control words are implementation-specific microinstructions, distinct from architectural RISC-V instructions.

2. Maximum data width in this NPU

Item	Width	Purpose
Activation / weight	Signed 8-bit	Compact neural values; loaded with sign extension.
Exact 8×8 product	Signed 16-bit	Fits an 8-bit signed activation × 8-bit signed weight; computed through repeated addition here.
Bias	Signed 32-bit	Added to the weighted-input sum.
Accumulator / output	32-bit	Accumulates weighted inputs and holds the post-ReLU result.
Architectural integer registers	32-bit	RV32 uses 32-bit integer registers and register arithmetic.

Maximum data width: 32 bits. A 16-bit compressed instruction does not mean that the datapath or registers are 16 bits wide.

3. Selected instructions and their purpose

Eight architectural instructions are the current candidate subset for the fixed one-neuron demonstration. This list must be checked against the final assembly program before the decoder is frozen.

Instruction	Purpose	Data / operation width
LB	Load signed 8-bit activation or weight into a 32-bit register using sign extension.	8-bit memory → 32-bit register
LW	Load the neuron's 32-bit bias.	32-bit memory → 32-bit register
SW	Store final 32-bit output to memory.	32-bit register → 32-bit memory
ADD	Add values for repeated addition and accumulate weighted-input sum.	32 + 32 → 32-bit result
ADDI	Initialize/update pointers and counters; set a register to zero when needed.	32-bit register + sign-extended immediate → 32-bit result
BNE	Repeat a loop while two registers differ, e.g. while a counter is nonzero.	Compare 32-bit registers; PC-relative branch
BGE	Signed comparison for ReLU control flow.	Signed 32-bit comparison; PC-relative branch
JAL	Jump to a terminal self-loop, e.g. `jal x0, 0`.	PC-relative control flow; 32-bit PC

Instruction count: 8 candidate instructions. There is no dedicated MUL instruction: multiplication is planned as repeated addition controlled by the program and microsequencer. Negative multipliers need explicit sign handling, which must be implemented and verified.

RV32C note: Not every instruction has a 16-bit encoding. Compression depends on the defined compressed forms and operand constraints. The fetch unit must identify instruction length and fetch the remaining 16 bits for a 32-bit instruction.

Current design summary

Target	RV32I instruction subset + RV32C encodings where supported
Multiplication	Repeated addition; no dedicated MUL instruction
Maximum data width	32 bits
Candidate instruction count	8
Control	MicroROM + microsequencer driving the standalone datapath